

Q1: with the advent of quantum computing, traditional public-key cryptography cryptosystems such as RSA and ECC are potentially vulnerable to shor's algorithm. Discuss the implication of quantum computing on the security of cryptographic protocols, and ~~propose~~ propose possible post-quantum cryptography algorithms that could replace RSA and ECC. How do these algorithms resist quantum cryptography?

Ans: quantum computing creates a serious threat to traditional public key cryptosystems like RSA and ECC. These systems are based on difficult mathematical problems such as integer factorization and discrete logarithm. However, shor's algorithm in quantum computing can solve

these problems very quickly. As a result, RSA and ECC may become insecure in the future.

The main implication^(QAQR) is that secure communication systems, digital signatures and key exchange protocols may break. Even encrypted data stored today could be decrypted later when powerful quantum computers become available.

To overcome this problem, post-quantum cryptographic (PQC) algorithms are proposed. Examples include lattice-based cryptography (like classic McEliece).

These algorithms resist quantum attacks because they are based on mathematical

Problems that currently have no efficient solution using quantum computers.

Q2: Design and implement a novel pseudo-Random Number Generator (PRNG) algorithm in python using the current timestamp, the process ID (`os.pid`) for added randomness, a modulus operation to constrain the output within a desired range.

⇒ To generate pseudo-random numbers we combine:

1. current timestamp: changes every moment
2. process ID (PID): different for each running process.
3. modulus operation: keeps output within a fixed range.

The timestamp and PID are used as a ~~seed~~ seed. They are applied to a mathematical formula and use modulus to the limit the result.

python implementation:

```
import time  
import os
```

```
def custom_rng(mod_range):
```

```
    timestamp = int(time.time() * 1000)
```

```
    pid = os.getpid()
```

```
    seed = timestamp ^ pid
```

```
    random_number = seed % mod_range
```

```
    return random_number
```

```
print(custom_rng(100))
```


Q:8 // compare traditional ciphers (such as the caesar cipher, vigenere cipher, and playfair cipher) with modern symmetric ciphers like AES and DES. Discuss the strengths and weaknesses of each type of cipher, including key length, encryption/decryption speed, and security against modern cryptanalytic techniques.

Ans: Traditional ciphers such as caesar cipher, vigenere cipher, and playfair cipher are classical encryption methods used in earlier times. They are simple substitution or transposition techniques and mainly operate on letters. These ciphers use very small keys and simple rules.

which make them easy to implement but weak against modern cryptanalysis. They can be easily broken using frequency analysis or brute-force attacks.

In contrast, modern symmetric ciphers like AES and DES use complex mathematical operations and work on binary data (bits). They use much larger key sizes (e.g., AES supports 128, and 256-bit keys), which makes them much more secure. DES uses a 56 bit key, which is now considered weak, but AES is currently very secure against modern attacks.

In terms of speed, traditional ciphers are simple but not practical for large

digital data, modern symmetric ciphers are designed for fast encryption and decryption of large amounts of data and are widely used in secure communication systems.

Therefore, traditional ciphers are mainly of historical importance, while modern symmetric ciphers provide strong security and efficiency in today's digital world.

Q4: Let S_4 be the symmetric group on the set $\{1, 2, 3, 4\}$. Define an action of S_4 on the set of 2 element subset of $\{1, 2, 3, 4\}$. Prove that this action is well defined, and compute

the size of the orbit and the stabilizer of the subset $\{1, 2\}$.

Ans:

Let S_4 be the symmetric group on the set $\{1, 2, 3, 4\}$.

Let X be the set of all 2-element subsets of $\{1, 2, 3, 4\}$. $|X| = 6$

So, $X = \{\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}\}$

for $\sigma \in S_4$ and $A = \{a, b\} \in X$ define:

$$\sigma \cdot \{a, b\} = \{\sigma(a), \sigma(b)\}$$

That is the permutation σ acts on each element of the subset.

Q5: Let $\text{GF}(2^2)$ be the finite field of order 4, constructed using the irreducible polynomial x^2+x+1 over $\text{GF}(2)$.

(i) show that $\text{GF}(2^2)$ forms a group multiplication.

(ii) verify whether the set of all non zero elements of $\text{GF}(2^2)$ is cyclic.

→

$\text{GF} \rightarrow$ finite field

$$\text{GF}(2) = \{0, 1\}$$

$$\text{GF}(p^n) = p = \text{prime number}$$

Ans: Think of $\text{GF}(2^2)$ as a set of rules for doing math with a special set of numbers. The elements in this field are $\{0, 1, x, x+1\}$.

(i) To be a group under multiplication, the set must follow four rules.

Because 0 has no multiplicative inverse ($0 \times \text{anything} \neq 0$), we exclude it and look at the set.

$$G = \{1, x, x+1\}$$

1. closure: multiply any two elements the result must still be in the set.

$$1 \cdot x = x$$

$$\rightarrow x \cdot (x+1)$$

given irreducible polynomial:
define $x^2 + x + 1 = 0$

1. closure: multiplying any two elements results in an element within the set.

$$\rightarrow x \cdot x = x^2 = x+1, x^2 = -(x+1) \text{ is same as } +(x+1) \text{ in } GF(2).$$

$$\rightarrow x \cdot (x+1) = x^2 + x + 1 = 2x + 1. \text{ Since } 2x \text{ is } 0 \text{ in } GF(2) \text{ This is equal to } 1 \text{ which is in the set.}$$

$$\rightarrow (x+1) \cdot (x+1) = x^2 + 2x + 1 = x^2 + 1 = -x = x \text{ which is in set}$$

All results are inside $\{1, x, x+1\}$,
so closure holds

2. Identity Element: There is a number
that changes nothing when multiplied.
That is 1 .

3. Associativity: multiplication order doesn't
matter. $x \cdot (1 \cdot (x+1)) = 1 \cdot (x \cdot (x+1))$.
This is true for polynomials.

4. Inverses: Every number has a partner
that multiplies with it to make 1 .

$$\rightarrow 1 \cdot 1 = 1$$

$$\rightarrow x(x+1) = x^2 + x = 1$$

Since all rules are satisfied. Therefore,
nonzero elements of $\text{GF}(2^2)$ form
a group under multiplication.

(ii)

verifying if the Non-zero
A group is cyclic if you can take
one element (the generator) and multiply
it by itself repeatedly to create every
single element in the group.

Let's test the element x :

1. $x^1 = x$.

2. $x^2 = x+1$.

1. ~~$(x+1) \cdot (x+1) = x^2 + 2x + 1 = x^2 + 1 = x$~~

3. $x^3 = x \cdot x^2 = x(x+1) = x^2 + x = 1$

Since x generated all the elements
in the set $\{x, x+1, 1\}$, the set is cyclic

Q6: Let $GL(2, R)$ be the general linear
group of 2×2 invertible matrices
over R . Show that the set of scalar
matrices forms a normal subgroup
of $GL(2, R)$. Construct the corresponding

factor group, and interpret its structure.

Ans:

Scalar matrices (Z) : matrices of the form $\begin{bmatrix} c & 0 \\ 0 & c \end{bmatrix}$ where $c \in \mathbb{R}$ and $c \neq 0$. Note that a scalar matrix can be written as cI , where I is the identity matrix.

To show Z is a normal subgroup, we must prove it is a subgroup and that it satisfies the normality condition: $CgZg^{-1} = Z$ for all $g \in GL(2, \mathbb{R})$.

ide H is subgroup.

1. Identity: The Identity matrix $I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ is in Z (set $c=1$).

2. closure: If $A = c_1 I$ and $B = c_2 I$ then
 $AB = (c_1 c_2) I$ which is in Z .

3. Inverses: If $A = c I$ then $A^{-1} = \frac{1}{c} I$,
which is in Z (since $c \neq 0$).

Normal: A subgroup is normal if it
commutes with all elements of the
larger group. Let $M = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ be any
matrix in $GL(2, \mathbb{R})$ and $S = \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix}$ be a
scalar matrix in Z .
Let compute MS and SM .

$$MS = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix} = \begin{bmatrix} ak & bk \\ ck & dk \end{bmatrix}$$

$$SM = \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} ka & kb \\ kc & kd \end{bmatrix}$$

Since $MS = SM$ scalar matrices commute
with all matrices in $GL(2, \mathbb{R})$.
Therefore Z is a normal subgroup.

construct the factor group $GL(2, \mathbb{R})/\mathbb{Z}$

The factor group consists of cosets of the form $m\mathbb{Z}$, where m is a matrix in $GL(2, \mathbb{R})$.

Two matrices A and B are in the same coset if $A = B\alpha$ for some scalar matrix $\alpha \in \mathbb{Z}$. This means $A = B(cI) = cB$.

Therefore, a coset $m\mathbb{Z}$ is the set of all non-zero scalar multiples of a matrix m . $m\mathbb{Z} = \{cm : c \in \mathbb{R}, c \neq 0\}$.

while $GL(2, \mathbb{R})$ represents all invertible linear transformations of a 2D plane, $PGL(2, \mathbb{R})$ represents the transformations on the projective line. It ignores the 'scale' or magnitude of the vectors and focuses only on their direction.

8.7

The Diffie Hellman key exchange is a method that allows two parties to create a shared secret key over a public channel. It is used to establish a secret key before encrypted communication begins.

How it works (step by step).

Suppose Alice and Bob want a shared key. They agree publicly on a large prime number p , a generator g . These are not secret.

Private keys: Alice chooses secret number a , Bob chooses secret number b . These are kept private.

Exchange public keys: Alice computes $A = g^a \text{ mod } p$ and sends A to Bob.

Bob computes $B = g^b \text{ mod } p$ and sends B to Alice.

IT24663

compute shared secret:

Alice computes: $k = B^a \bmod p$

Bob computes: $k = A^b \bmod p$

Both get: $k = g^{ab} \bmod p$

Now they share the same secret key

Application in secure communication

Diffie-Hellman is used in:

→ HTTPS → VPNs → secure messaging
apps → secure email.

It allows two users to create a secret key even if an attacker listens to the entire conversation.

Secure communication depends on the discrete logarithm problem.

Given: $g^a \bmod p$. It is extremely difficult to find a when p is very large. This difficulty protects the

private keys.

MITM (man in the middle): Diffie-Hellman is vulnerable to MITM attacks because it lacks authentication. An attacker can intercept Alice's public key and send her own to Bob, effectively establishing two separate shared secrets ($g^a e$ and $g^b e$). Attacker can then decrypt, read, and re-encrypt messages between them. To prevent this, DH is usually paired with digital signatures or certificates.

if the prime p not significantly larger (less than 2048 bits). the protocol is insecure against 1. brute force / Index calculus: Attackers can use specialized algorithms to solve the DLP in a reasonable timeframe.

IT29663

2. precomputation Attacks: Attackers precalculate tables (like Rainbow tables for hashes) to quickly map public values to private exponents.

Application:

- TLS/SSL: To negotiate session keys for HTTPS website traffic
- SSH: for secure remote login.
- IPsec: for securing data flow in virtual private networks (VPNs).

8.8

Let H and K be two subgroups of a group G , we want to prove that their intersection, $H \cap K$, is also a subgroup of G .

1. Identity element:

Since H is a subgroup, the identity element $e \in H$.

Since K is a subgroup, the identity element $e \in K$.

Because e is in both H and K , it must be in their intersection: $e \in (H \cap K)$.

2. Closure: Take any two elements a and b in $H \cap K$.

→ Since $a, b \in H$ and H is a subgroup, then $ab \in H$.

→ Since $a, b \in K$ and K is a subgroup, then $ab \in K$.

Because the product ab is in both H and K , it follows that $ab \in (H \cap K)$.

3. Inverses: Take any element a in $H \cap K$.

→ Since $a \in H$ and H is a subgroup, the inverse $a^{-1} \in H$.

→ $a \in K$ and K is a subgroup, the inverse $a^{-1} \in K$. Because a^{-1} is in both, $a^{-1} \in (H \cap K)$.

Since $H \cap K$ contains the identity, is closed under the group operation and contains inverses for all its elements, it is a subgroup of G .

Example:

Let G be the group of integers under addition $(\mathbb{Z}, +)$.

Let H be the subgroup of multiples

Let $H = 2\mathbb{Z} = \{ \dots, -4, -2, 0, 2, 4, 6, \dots \}$

and $K = 3\mathbb{Z} = \{ \dots, -6, -3, 0, 3, 6, 9, \dots \}$

$H \cap K = 6\mathbb{Z} = \{ \dots, -12, -6, 0, 6, 12, \dots \}$

$6\mathbb{Z}$ is valid subgroup of $(\mathbb{Z}, +)$

8.9

A ring is commutative if the multiplication operation is independent of the order of the elements: $a \cdot b = b \cdot a$.

IT24663

→ In $\mathbb{Z}_n$ multiplication is defined as:

$$ab = (axb)$$

we know that for any two integers a and b , the standard multiplication is commutative $a \times b = b \times a$, therefore: $(axb)(\text{mod } n) = (bxa)(\text{mod } n)$.

This shows that $a \cdot b = b \cdot a$ for all elements in $\mathbb{Z}_n$. Thus, $\mathbb{Z}_n$ is a commutative ring.

identifying zero divisors: A zero divisor is a non-zero element a such that there exists a non-zero b where $a \cdot b = 0 (\text{mod } n)$.

$\mathbb{Z}_n$ has ~~has~~ zero element divisors if and only if n is composite. if n is composite it can be factored into $n = r \cdot s$ where $1 < r, s < n$.

In $\mathbb{Z}_n$, both $[r]_n \neq 0$ and $[s]_n \neq 0$ yet.

II29663

$$\text{thi } [2]_n \cdot [3]_n = [6]_n = [0]_n = 0$$

Example $(\mathbb{Z}_6)$: $[2] \neq 0$ and $[3] \neq 0$, but
 $[2] \cdot [3] = [6] = 0 \pmod{6}$. Thus 2 and 3 are
 zero divisors.

A zero divisor is a non-zero element
 a such that there exists another
 non-zero element b where $a \cdot b = 0$
 $a \cdot b \equiv 0 \pmod{n}$.

For a ring to be a field, every non-
 element must have a multiplicative
 inverse (means an element a^{-1}
 such that $a \cdot a^{-1} \equiv 1 \pmod{n}$)
 $\mathbb{Z}_n$ is a field if and only if n is a
 prime number. Because every number
 $a \in \{1, 2, \dots, n-1\}$ is coprime to n
 $(\gcd(a, n) = 1)$. According to number
 theory, if $\gcd(a, n) = 1$, then a has a
 modular multiplicative inverse.

8.10

The Data Encryption Standard is a symmetric ~~book~~ block cipher developed in the 1970s. Block size 64 bits, key size 56 bits. Based on Feistel structure used for government and commercial encryption. It was once widely used but ~~now~~ is now considered insecure.

One of the most significant weaknesses of DES is its short key length. DES uses a 56 bit key. Mathematically, this means there are only 2^{56} possible keys. Today this is too small. Modern computers can try all possible keys within a short time.

A brute force attack involves trying every possible key until for about \$250,000 and broke a DES key in 56 hours.

IT2963

In 1999 distributed computing broke a key in 22 hours. Today a modern high-end computer or a cluster of GPUs can crack a DES key in a matter of minutes or hours. Because the key is so short, any attacker with moderate resources can recover the plaintext without needing to find a 'flaw' in the math itself.

DES was replaced by Advanced Encryption Standard. AES was selected by NIST in 2001. AES was designed specifically to fix the small room left by DES: structural and mathematical flaws. AES has 128, 192, or 256 bits key size, 128 block size, substitution-permutation network structure.

IT29663

key improvements in AES:

1. massive key space: Even at the smallest key size (128 bit), the number of possible keys is 2^{128} . To put this in perspective, 2^{128} is roughly 3.4×10^{38} . Even a "supercomputer" that could check a trillion keys per second and could take longer than the age of the universe to ~~crack~~ crack it.

2. Increase block size: DES uses 64 bit blocks, which makes it vulnerable on large amounts of data. AES uses 128 bit blocks, which significantly increases resistance to such collisions.

3. Resistance to cryptanalysis: AES was built using a different mathematical structure (the substitution-permutation network) that provides better diffusion and "confusion", making it much more resistance to linear and differential cryptanalysis than the

the aging Feistel structure of DES

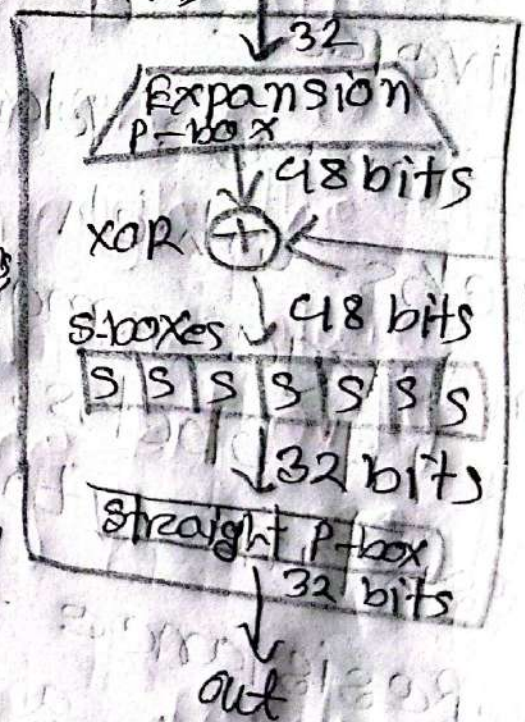
8.11

(i)

The Feistel structure of DES was actually designed with differential cryptanalysis in mind (though the public didn't know this until the 1990s)

1. The Role of S-boxes: In

In DES, the only non-linear part is the S-boxes. All other operations (permutations, XORs) are linear. Differential cryptanalysis focuses on how difference propagate through these S-boxes.



2. Hardened Design: The IBM and NSA designers specifically tuned the bit-patterns inside the 8 S-boxes so the probability of a specific input

IT24963

difference leading to a specific output difference was kept as low as possible

3. The 'f' function: Because DES uses a Feistel structure, only half the block is modified in each round. The output of the f function (which contains the S-boxes) is XORed with the other half. This structure ensures that even if an attacker finds a 'good' differential for one round, it is difficult to keep that differential 'active' or useful through all 16 rounds

(ii)
AES is more resistant.
AES abandoned the Feistel structure for a substitution-permutation network (SPN). It is more resistant to differential cryptanalysis due

to its "wide Trail strategy"

In DES, ~~s-boxes~~

1. SubBytes: In DES, s-boxes were somewhat "random" tables. In AES

The subBytes step uses a sophisticated mathematical formula (the inverse over GF(2^8)). This ensures that the maximum probability of any differential transition is extremely low (2^{-6}), making it much harder to find a path through the cipher.

2. Diffusion (Shift Rows and Mix Columns):

In DES, a change in one bit only affects a small part of the next round. In AES:

→ Shift Rows moves bytes to different columns.

→ Mix Columns ensures that a change in just one byte of a column

spreads to all four bytes of that column in the next round.

This is called the wide trail strategy. It ensures that a differential "trial" quickly becomes so complex (involving so many "active" s-boxes) that the probability of the attack succeeding becomes mathematically negligible.

3. Full Block processing; unlike DES, which only processes 32 bits of the 64 bit block per round, AES processes the entire 128 bit block in every single round. This means the 'confusion and diffusion' happen twice as fast, requiring an attacker to track many more variables simultaneously.

8:12

Ans: The Extended Euclidean Algorithm (EEA) is the standard method for finding the modular multiplicative inverse. It extends the basic Euclidean Algorithm by expressing that GCD as a linear combination of the two input integers.

The Extended Euclidean Algorithm finds integers x and y such that $ax + by = 1$ if $\gcd(a, n) = 1$, then x is the modular inverse of a modulo n .

we want to find: $a^{-1} \pmod{n}$

that means find integer x such that $ax \equiv 1 \pmod{n}$

this is possible only if: $\gcd(a, n) = 1$

Example.

Find inverse of: $a=7, n=26$

we want $7x \equiv 1 \pmod{26}$

Apply Euclidean Algorithm:

$$26 = 3 \times 7 + 5$$

$$7 = 1(5) + 2$$

$$5 = 2(2) + 1$$

$$2 = 2(1) + 0$$

so $\gcd(7, 26) = 1$ so inverse.

Back substitution,

$$1 = 5 - 2(2)$$

$$= 5 - 2(7 - 1(5))$$

$$= 5 - 2(7) + 2(5)$$

$$= 3(5) - 2(7)$$

$$\text{Now, } 5 = 26 - 3(7)$$

$$\text{so, } 1 = 3(5) - 2(7)$$

$$= 3(26 - 3(7)) - 2(7)$$

$$= 3(26) - 9(7) - 2(7)$$

$$= 3(26) - 11(7)$$

$$\text{so, } 1 = (-11)(7) + 3(26)$$

Thus $-11 = 15 \pmod{26}$

So modular Inverse 7^{-1} is $= 15$.

In RSA after selecting public exponent e , the private key d is computed as the modular inverse of e modulo $\phi(n)$. This is done using the Extended Euclidean Algorithm

The RSA algorithm works like this

1. choose primes p, q ;

2. compute: $n = p \cdot q$.

$$\phi(n) = (p-1)(q-1)$$

choose public key e such that:

$$\gcd(e, \phi(n)) = 1$$

compute private key d such that:

$$ed = 1 \pmod{\phi(n)}$$

This means $d = e^{-1} \pmod{\phi(n)}$.

This modular inverse is found using the Extended Euclidean Algorithm.

So EEA is used to compute the private key.

In modern systems RSA uses 2048 bit or 4096 bit numbers. $\phi(n)$ becomes extremely large. If algorithm is slow. Then key generation becomes impractical and Encryption systems become inefficient. The Extended Euclidean Algorithm runs in $O(\log n)$ which is very efficient even for large numbers. This makes RSA practical for HTTPS, Digital Signatures, secure banking.

Ans: Q.13

(i) Electronic codebook (ECB) mode encrypts each block independently. For a sequence of plaintext blocks $P_1, P_2, \dots, P_m$, the ciphertext C_i is

$$C_i = E_k(P_i)$$

if $P_i = P_j$ ($i \neq j$) then
 $C_i = E_K(P_i) = E_K(P_j) = C_j$

Thus $P_i = P_j \Rightarrow C_i = C_j$

so for highly redundant data, identical plaintext blocks produce identical ciphertext blocks. Hence, structural information about the plaintext is leaked.

Therefore, ECB is insecure because it does not provide semantic security and reveals patterns.

(ii)

Cipher block chaining (CBC) uses the previous ciphertext to encrypt the current block.

Recurrence Relations

Encryption: $C_i = E_K(P_i \oplus C_{i-1})$ with $C_0 = IV$

Decryption: $P_i = D_K(C_i) \oplus C_{i-1}$

suppose ciphertext block c_i is corrupted. Then $P_i = D_K(C_i) \oplus C_{i-1}$ so P_i becomes completely corrupted.

Next block: $P_{i+1} = D_K(C_{i+1}) \oplus C_i$

(since c_i is corrupted, only some bits of P_{i+1} are affected) blocks after that are unaffected. Thus error propagation in CBC decryption is limited to two blocks only

then block P_{i+2} : since C_{i+1} and C_{i+2} are clean, P_{i+2} is recovered perfectly.

The error is self healing and only impacts the current and the immediately following block.

8.14

An linear feedback shift register is a stream cipher component that generates bits using a linear recurrence relation over $\text{GF}(2)$.

General form: $s_{t+n} = c_1 s_{t+n-1} + c_2 s_{t+n-2} + \dots + c_m s_{t+n-m}$

Each new bit is a linear combination of previous bits.

In stream cipher $c_t = p_t \oplus k_t$ where k_t = keystream bit, if some plain text bits are known, the attacker can compute the corresponding keystream bits. These bits satisfy a system of linear

equations. means $s_{t+n} = \sum c_i s_{t+1}$ This is the form of linear equation.

With sufficient known bits, the attacker can solve the equations to recover the feedback coefficients and predict future keystream bits. Thus, linearity makes LFSRs vulnerable to known-plaintext & to attacker.

This vulnerability can be mitigated by break linearity.

1. Method 1: Nonlinear combination function. Use multiple LFSRs and combine outputs using a nonlinear function.

$k_t = f(s_t^{(1)}, s_t^{(2)}, s_t^{(3)})$ where f is nonlinear. This prevents forming simple linear equation.

method: Nonlinear filterz function.
Apply a nonlinear Boolean function to
LFSR state. $k_t = f(s_t, s_{t+1}, \dots)$

if f is nonlinear, attacker cannot solve
using linear algebra.

Nonlinearity prevents the formation
of solvable linear equations, thereby
improving security.

S.A. 15

Ans:

(i)

Shannon's Definition of perfect secrecy:

A cryptosystem has perfect secrecy if the probability distribution of the plaintext is independent of the ciphertext. Mathematically, for every plaintext $m \in M$ and every ciphertext $c \in C$:

$$P(M=m|C=c) = P(M=m).$$

This means that even if an attacker intercepts the ciphertext c , they gain zero additional information about the original message m . The "after seeing the ciphertext" probability is exactly the same as the "before-seeing" probability.

(ii)

One-Time pad (OTP) Achieves perfect

In OTP, the encryption is $c = m \oplus k$.
Assume the key k is chosen uniformly at random from the set of all possible keys.

1. probability of specific ciphertext: for any m and c , there is exactly one key k such that $k = m \oplus c$.

2. since k is uniform: The probability of picking that specific key is

$$P(K=k) = \frac{1}{|K|}.$$

2. using Bayes' Theorem:

$$P(m=m|c=c) = \frac{P(c=c|m=m) \cdot P(m=m)}{P(c=c)}.$$

4. Because $P(C=c|M=m)$ is just the required key k was chosen, it is $\frac{1}{|K|}$.

3. Since this value is the same for every m , the ciphertext c provides no bias. The $P(C=c)$ term cancels out the key probability, leaving

$$P(M=m|C=c) = P(M=m)$$

Thus the one-time pad is mathematically unbreakable.

(iii)
Despite being 'perfect', the one-time pad is rarely used in large-scale systems (like the internet) for three main reasons:

1. Key length constraint: Shannon proved that for perfect secrecy, the key must be at least as long as the message ($|K| \geq |M|$). To encrypt a 10GB video file, you need a 10GB key.

2. The key distribution problem: If you have a secure way to send a 10GB secret key to someone, you could have just used that same secure way to send the 10GB message itself. It creates a "chicken and egg" problem.

3. Key management (Reuse): The "one-time" part is literal. If you use the same key twice ($C_1 = m_1 \oplus K$ and $C_2 = m_2 \oplus K$)

an attacker can XOR the ciphertexts.

$$c_1 \oplus c_2 = m_1 \oplus m_2.$$

This removes the key entirely and leaves the messages vulnerable to frequency analysis.

$$8: \underline{\underline{16}}$$

given LCG formula

$$x_{n+1} = (ax_n + c) \bmod m$$

we must choose specific values for multiplier a , increment c , modulus m , seed $x_0 = 7$.

$$\text{let } a=7, c=3, m=16.$$

so the recurrence becomes:

$$x_{n+1} = (5x_n + 3) \bmod 16$$

$$x_0 = 7$$

$$\text{so } x_1 = (5 \times 7 + 3) \bmod 16$$

$$= 38 \bmod 16$$

$$= 6$$

$$\begin{aligned}x_2 &= (5 \times 6 + 3) \bmod 16 \\&= 33 \bmod 16 \\&= 1\end{aligned}$$

$$\begin{aligned}x_3 &= (5 \times 1 + 3) \bmod 16 \\&= 8\end{aligned}$$

$$\begin{aligned}x_4 &= (5 \times 8 + 3) \bmod 16 \\&= 43 \bmod 16 = 11\end{aligned}$$

$$\begin{aligned}x_5 &= (5 \times 11 + 3) \bmod 16 \\&= 58 \bmod 16 \\&= 10\end{aligned}$$

The first 5 numbers are called seed
are: 6, 1, 8, 11, 10

So the sequence starting from $x_0 = 7$
is: 7, 6, 1, 8, 11, 10, ...

8.17

A Ring is a set R equipped with two binary operations. Addition $(+)$, multiplication $(\cdot)$. such that

(i) $(R, +)$ is an abelian group:
closure under addition, Associativity, Additive identity 0 exists, Additive inverse exist, commutativity of addition.

2. monoid under multiplication: $(R, \cdot)$ must be associative and usually has a multiplicative identity (1) .

3. Distributivity: multiplication must distribute over addition:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

$$(b + c) \cdot a = (b \cdot a) + (c \cdot a)$$